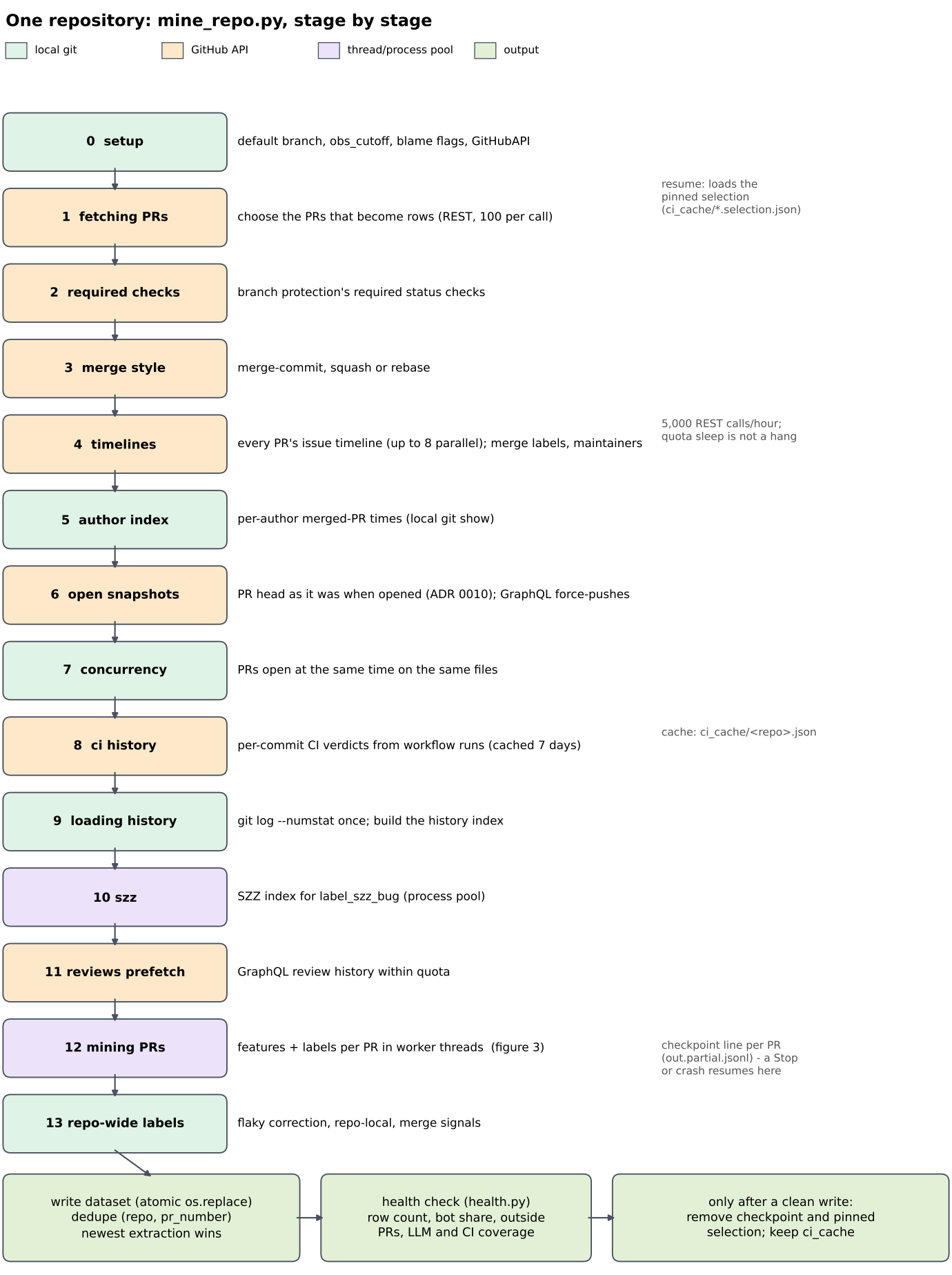
One repository: mine_repo.py, stage by stage
local git
GitHub API
thread/process pool
output
0 setup
default branch, obs_cutoff, blame flags, GitHubAPI
1 fetching PRs
choose the PRs that become rows (REST, 100 per call)
resume: loads the pinned selection (ci_cache/*.selection.json)
2 required checks
branch protection's required status checks
3 merge style
merge-commit, squash or rebase
4 timelines
every PR's issue timeline (up to 8 parallel); merge labels, maintainers
5,000 REST calls/hour; quota sleep is not a hang
5 author index
per-author merged-PR times (local git show)
6 open snapshots
PR head as it was when opened (ADR 0010); GraphQL force-pushes
7 concurrency
PRs open at the same time on the same files
8 ci history
per-commit CI verdicts from workflow runs (cached 7 days)
cache: ci_cache/<repo>.json
9 loading history
git log --numstat once; build the history index
10 szz
SZZ index for label_szz_bug (process pool)
11 reviews prefetch
GraphQL review history within quota
12 mining PRs
features + labels per PR in worker threads (figure 3)
checkpoint line per PR (out.partial.jsonl) - a Stop or crash resumes here
13 repo-wide labels
flaky correction, repo-local, merge signals
write dataset (atomic os.replace)
dedupe (repo, pr_number)
newest extraction wins
health check (health.py)
row count, bot share, outside PRs, LLM and CI coverage
only after a clean write:
remove checkpoint and pinned selection; keep ci_cache